case5.java

```
package com.example;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.ArrayList;
import java.util.List;

/**
 * Case Study 5: Online Complaint Management System
 *
 * Problem Statement:
 * A university wants an application through which students can register complaints
 * about classrooms, Wi-Fi, or library facilities. The system should assign each
 * complaint a status.
 *
 * Concepts: REST API, POST, GET, request validation, status tracking
 *
 * Requirements:
 * 1. Accept a complaint description and category.
 * 2. Assign a complaint ID.
 * 3. Set the initial status to PENDING.
 * 4. Retrieve all registered complaints.
 */

@SpringBootApplication
public class case5 {

    public static void main(String[] args) {
        SpringApplication.run(case5.class, args);
        System.out.println("Online Complaint Management System is running on http://localhost:8080");
    }
}

// ---------------- MODEL CLASS ----------------
class Complaint {
    private int complaintId;
    private String description;
    private String category; // Classrooms, Wi-Fi, Library, etc.
    private String status;     // Initial status: PENDING

    // Default constructor
    public Complaint() {
        this.status = "PENDING";
    }

    // Parameterized constructor
    public Complaint(String description, String category) {
        this.description = description;
        this.category = category;
        this.status = "PENDING";
    }

    public Complaint(int complaintId, String description, String category, String status) {
        this.complaintId = complaintId;
        this.description = description;
        this.category = category;
        this.status = status;
    }

    // Getters and Setters
    public int getComplaintId() {
        return complaintId;
    }

    public void setComplaintId(int complaintId) {
        this.complaintId = complaintId;
    }

    public String getDescription() {
        return description;
    }

    public void setDescription(String description) {
        this.description = description;
```

```
    }

    public String getCategory() {
        return category;
    }

    public void setCategory(String category) {
        this.category = category;
    }

    public String getStatus() {
        return status;
    }

    public void setStatus(String status) {
        this.status = status;
    }

    @Override
    public String toString() {
        return "Complaint [complaintId=" + complaintId + ", description=" + description +
               ", category=" + category + ", status=" + status + "]";
    }
}

// ---------------- REST CONTROLLER ----------------
@RestController
@RequestMapping("/complaints")
class ComplaintController {

    // In-memory list to store complaints
    private List<Complaint> complaintList = new ArrayList<>();

    // Counter to assign auto-increment complaint ID
    private int idCounter = 1;

    // Endpoint 1: Register a new complaint (POST)
    // URL: POST http://localhost:8080/complaints
    /*
     * Sample JSON Body:
     * {
     *     "description": "Wi-Fi is not connecting in Library 2nd floor",
     *     "category": "Wi-Fi"
     * }
     */
    @PostMapping
    public ResponseEntity<?> registerComplaint(@RequestBody Complaint complaint) {
        // Concept: Request Validation
        if (complaint.getDescription() == null || complaint.getDescription().trim().isEmpty()) {
            return ResponseEntity.status(HttpStatus.BAD_REQUEST)
                    .body("Validation Error: Complaint description cannot be empty.");
        }

        if (complaint.getCategory() == null || complaint.getCategory().trim().isEmpty()) {
            return ResponseEntity.status(HttpStatus.BAD_REQUEST)
                    .body("Validation Error: Complaint category (e.g., Classroom, Wi-Fi, Library) cannot
be empty.");
        }

        // Assign unique complaint ID
        complaint.setComplaintId(idCounter++);

        // Set initial status to PENDING
        complaint.setStatus("PENDING");

        // Save to ArrayList
        complaintList.add(complaint);

        return ResponseEntity.status(HttpStatus.CREATED).body(complaint);
    }

    // Endpoint 2: Retrieve all registered complaints (GET)
    // URL: GET http://localhost:8080/complaints
    @GetMapping
    public List<Complaint> getAllComplaints() {
        return complaintList;
    }

    // Endpoint 3: Status Tracking - Get complaint details by ID (GET)
    // URL: GET http://localhost:8080/complaints/{id}
    @GetMapping("/{id}")
    public ResponseEntity<?> getComplaintById(@PathVariable int id) {
```

```
        for (Complaint c : complaintList) {
            if (c.getComplaintId() == id) {
                return ResponseEntity.ok(c);
            }
        }
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
                .body("Complaint with ID " + id + " not found.");
    }

    // Endpoint 4: Status Tracking - Update status of a complaint (PUT)
    // URL: PUT http://localhost:8080/complaints/{id}/status?status=RESOLVED
    @PutMapping("/{id}/status")
    public ResponseEntity<?> updateComplaintStatus(@PathVariable int id, @RequestParam String status) {
        for (Complaint c : complaintList) {
            if (c.getComplaintId() == id) {
                c.setStatus(status.toUpperCase());
                return ResponseEntity.ok("Status of Complaint ID " + id + " updated to " + c.getStatus(
));
            }
        }
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
                .body("Complaint with ID " + id + " not found.");
    }
}
```